

QoE Mobile Network Monitoring Application

System Modelling and Design - Task 4

CEF440: Programming and Mobile Programming

Group 4

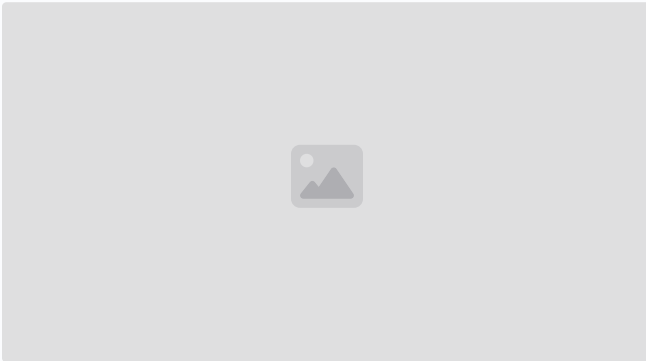
University of Buea

May 2026

System Modelling Overview

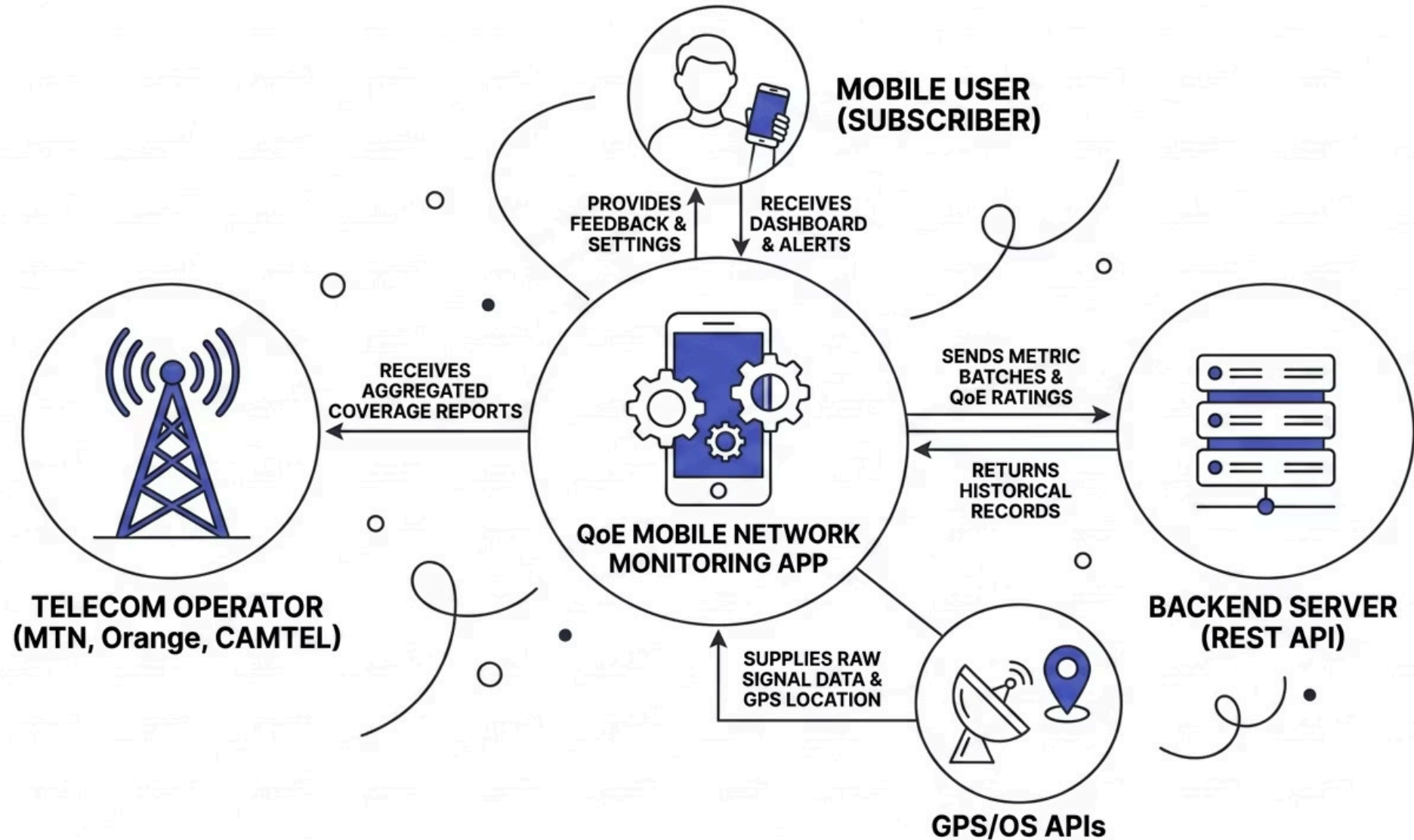
- System modelling translates requirements into concrete architectural blueprints
- This presentation covers 6 formal UML diagrams
- Each diagram provides a different view of the system
- Diagrams are traceable to validated requirements from Task 3

Context Diagram



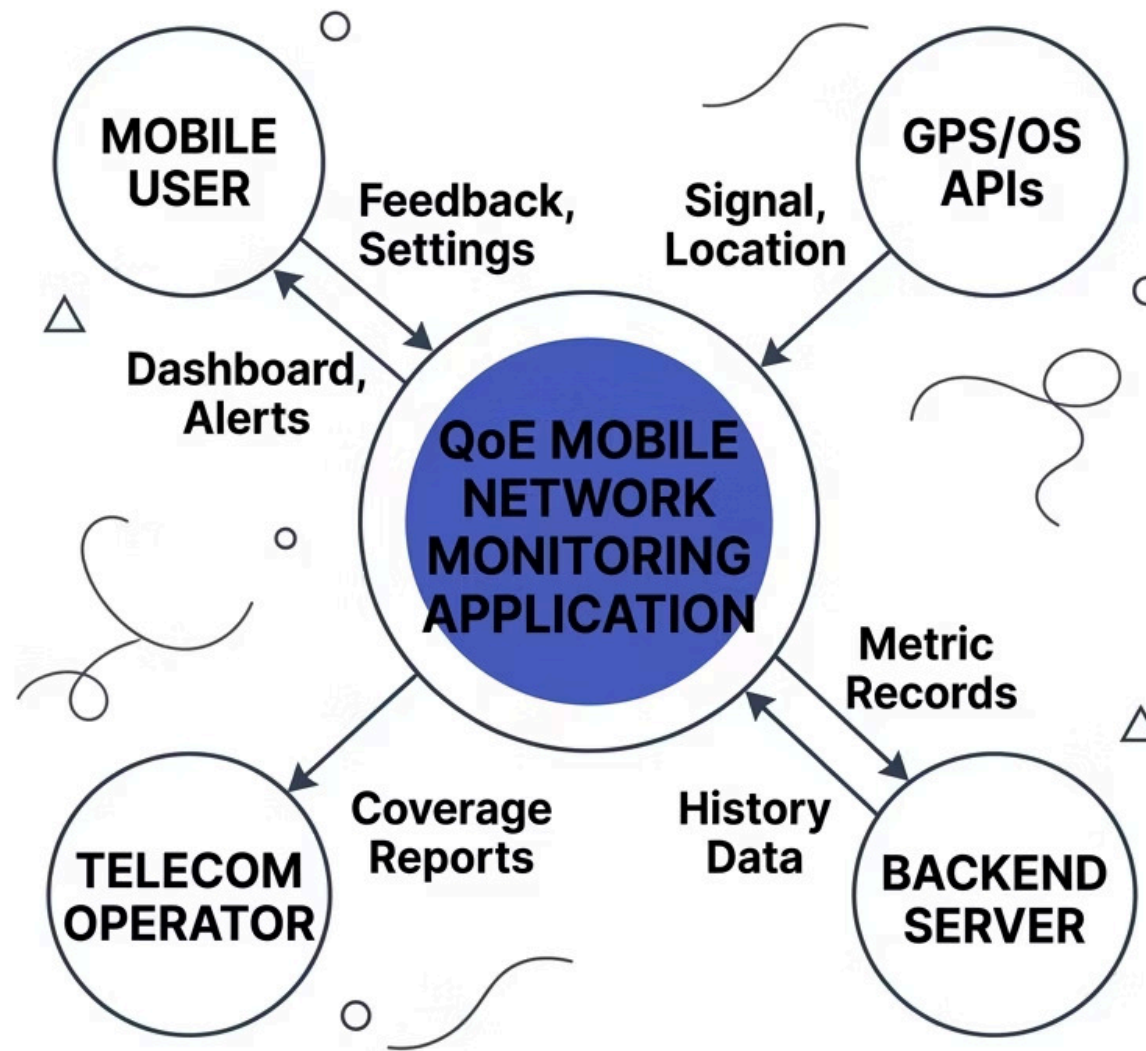
- Defines system boundary and external entities
- 4 external entities: Mobile User, GPS/OS APIs, Backend Server, Telecom Operator
- Shows top-level data flows crossing the system boundary

Context Diagram



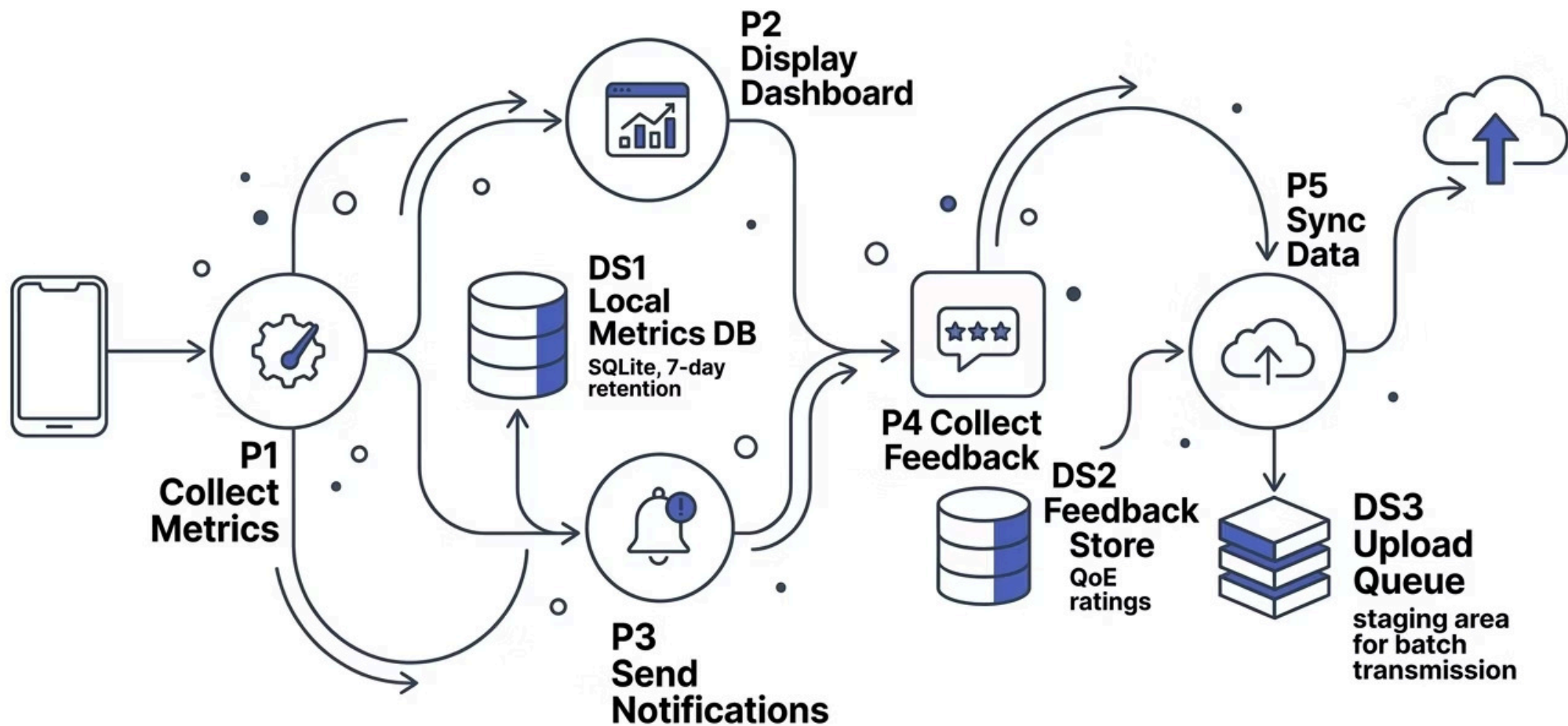
The context diagram defines the system boundary and identifies all external entities the system interacts with

Data Flow Diagram - Level 0



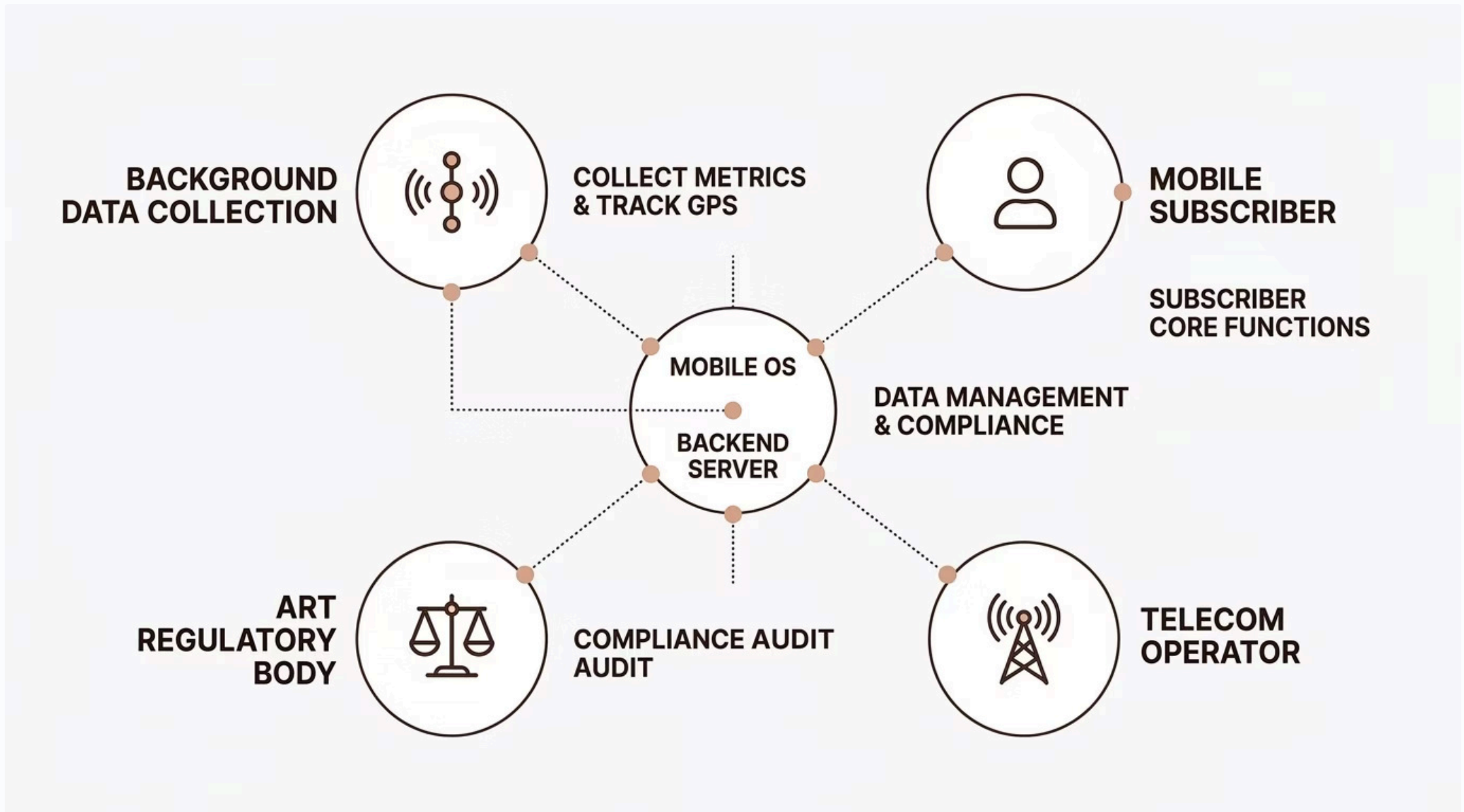
- Expands context diagram with explicit system boundary
- Single high-level process (Process 0): QoE Monitoring Application
- 6 major data flows: Signal/Location in, Feedback/Settings in, Dashboard/Alerts out, Metrics to server, History from server, Coverage reports to operator

Data Flow Diagram - Level 1



- Decomposes Process 0 into 5 internal processes (P1-P5)
- P1: Collect Metrics | P2: Display Dashboard | P3: Send Notifications | P4: Collect Feedback | P5: Sync Data
- 3 data stores: DS1 (Local Metrics DB), DS2 (Feedback Store), DS3 (Upload Queue)
- Shows complete internal data flow architecture with offline-first design

Use Case Diagram



- 5 actors: Mobile Subscriber, Mobile OS, Telecom Operator, Backend Server, ART Regulatory Body
- 24 use cases organized into 6 functional groups
- Groups: Subscriber Core Functions, Background Data Collection, Data Management & Compliance, Reporting & Analytics, System Sub-Functions, Extension Points
- All use cases traceable to validated requirements

Important Use Cases



UC1: Register & Grant Consent

Explicit consent screen at first launch



UC2: View Real-Time Dashboard

Live display of network metrics (refreshes every 2 seconds)



UC7: Collect Signal Metrics

Background collection every 60 seconds



UC4: Submit QoE Feedback

1-5 star rating, max 3 prompts per day



UC12: Sync Data to Server

Batch upload over HTTPS/TLS 1.2+



UC6: Speed Test

User-initiated active bandwidth measurement

These 6 use cases represent the core functionality of the system

Sequence Diagram

Flow 1: App Launch & Initialization

User opens app, background worker registered,
permissions granted

Flow 3: Satisfaction Prompt & Feedback Logging

User submits QoE rating (max 3/day), stored with
timestamp

Flow 5: Offline Data Synchronization

Background service detects connectivity, uploads batch
over TLS 1.2+

Flow 2: Periodic Background Metric Collection

Background service collects metrics every 60 seconds,
stores to SQLite

Flow 4: Real-Time Dashboard Display

User opens dashboard, app queries SQLite, renders live
metrics

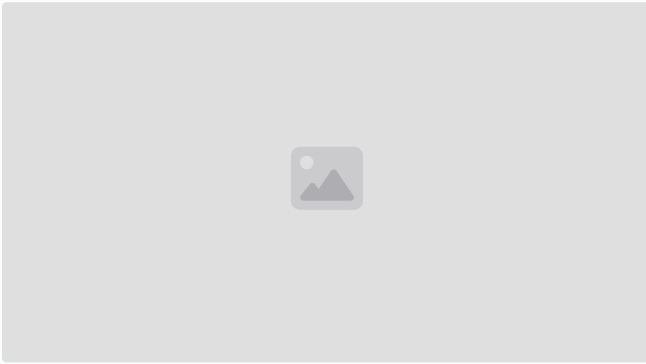
Flow 6: Active Speed Test

User triggers test, app measures latency/bandwidth,
stores result

Class Diagram

- Core classes: User, NetworkMetric, LocationData, Feedback, MonitoringService, StorageManager, Dashboard
- MonitoringService: Background collection engine
- StorageManager: AES-256 encryption and offline queue management
- Privacy-by-design: LocationData only associated with NetworkMetric if consent granted
- Supports 4-tier architecture: Client → Network → Backend → Stakeholders

Deployment Diagram



- 4-tier architecture: Mobile devices → HTTPS network → Backend server → Stakeholder dashboards
- Mobile nodes: Android 10+ and iOS 16+ with React Native
- Backend: Node.js/Express with PostgreSQL database
- Communication: HTTPS/TLS 1.2+ on Port 443, PostgreSQL on Port 5432

Key Design Decisions



Offline-First Architecture

All metrics written to SQLite before upload; zero data loss guarantee



AES-256 Encryption

All data encrypted at rest on device



Consent-Based GPS

GPS collection gated by explicit user consent; only district-level data stored



Background Execution

Platform-specific APIs (WorkManager/BackgroundTasks) for guaranteed execution



Battery Optimization

Automatic throttling when approaching 3% per hour limit



Privacy Compliance

Anonymization before transmission, 7-day local retention, audit trails

 These decisions directly address the 68.8% of survey respondents who cited privacy as primary concern

Technical Constraints

iOS Limitations:

- No raw dBm access; use qualitative "Signal Bars" instead
- Background tasks limited to ~15-minute intervals

Android Advantages:

- Full access to RSRP, RSSI, SINR metrics
- WorkManager guarantees periodic execution

Offline Queue & Battery:

- SQLite limited to 15MB or 7 days (whichever comes first)
- Background service: $\leq 3\%$ battery drain per hour, $\leq 5\text{MB}$ data per day

Conclusion

Summary:

- 6 formal UML diagrams completed
- 24 use cases fully traceable to requirements
- 4-tier architecture designed for scalability and security

Key Achievements:

- Privacy-by-design: Consent-gated, encrypted, anonymized
- Offline-first: Zero data loss guarantee
- Battery-conscious: Automatic throttling

Next Steps:

- Design ready for implementation
- Development team can proceed directly from these models
- Sequence diagram provides critical path for sprint planning

Questions & Answers

Thank You

Questions & Discussion

- CEF440: Programming and Mobile Programming
- University of Buea, Faculty of Engineering and Technology
- Group 4